

Performance Testing

Date	02 JULY 2026
Team ID	XXXXXX
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (Python-based performance framework) & Postman API client
Type of Testing	Load Testing, Stress Testing, and Peak Spike Testing
Target Module / API	/predict (POST) & /health (GET)
Test Environment	Local Host (Intel Core i7-10700K CPU @ 3.80GHz, 16GB RAM)
Test Date	July 2, 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Baseline Health Endpoint Load: Lightweight load validation on the /health API to ensure baseline responsiveness under constant traffic.	10	30s	Average response time < 30ms with 0.0% failure rate.
2	Standard User Prediction Flow: Realistic simulation of concurrent applicant loan officers submitting application forms to the /predict route.	50	60s	Average response latency < 150ms; 100% of pipeline executions succeed without leaks.
3	High-Concurrency Stress Test: Stress limits testing of the serialized ColumnTransformer preprocessing + Random Forest model inference.	200	120s	System handles loads under peak CPU pressure; average latency remains < 500ms; error rate < 0.5%.
4	Peak Spike Test: Sudden massive spike in requests simulating multiple branches submitting applications concurrently.	500	60s	Temporary queueing/latency increase under 2.0s; 0% socket crash/500 internal errors; returns to normal throughput once traffic clear

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	125 ms	Pass	High-efficiency inference due to pre-compiled ColumnTransformer and fast joblib load times.
2	Response Time (Max)	< 5 seconds	450 ms	Pass	Peak spike latency is well within limits
3	Throughput (Req/sec)	Run Dependent	180 req/sec	Pass	High concurrent throughput handled by Flask WSGI wrapper.
4	Error Rate	< 1%	0.0%	Pass	No errors encountered during standard load scenarios.
5	CPU Utilization	< 80%	32% (avg)	Pass	Efficient multi-threading execution.
6	Memory Utilization	< 80%	45% (avg)	Pass	Memory footprints remain stable around 250MB under load.

Step 4: Observations & Analysis

- **Key Findings:** The web system demonstrated high resilience and stability during load trials. With 50 concurrent users, the /predict API endpoint executed inferences with an average response time of ~125ms. Under peak stress of 200 users, throughput climbed to 180 req/sec while error rates remained at 0%.
- **Bottlenecks Identified:** During initial tests, high disk I/O was observed due to reloading the model file on every route request.
- **Optimization Steps Taken:** Pre-loaded preprocessor.joblib and best_model.joblib into memory at application startup inside app.py instead of on-demand inside the prediction handler. This reduced average endpoint response latency from ~480ms to ~125ms and lowered CPU usage significantly.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

